程序设计综合实验

第 2 周：斐波那契、特征值与排序

黄韬
taohonker@gmail.com
昆明理工大学·系统科学系

2026-09-23

课前

本周提要

- 从 Fibonacci 数列的计算看递归思想与算法复杂性
- 在数组上实现几种排序算法
- 从快速排序算法看分治思想

关于讲义与课程内容

- 讲义是自编的，不正式，勿外传
- 讲义里的 Julia 代码都在 julia 1.12 上跑过，**计时数据是本机实测值**，换一台机器数字会变，量级与增长趋势不变

Q 1 AI 1

关于作业

- 理论课每周课后布置一次 Homework，**统一次周周二晚上提交**
- 实验课共有 6 个 Lab、十余个检查点（checkpoint），平均 1-2 周提交一次
- 理论课 Homework 与实验课 Lab 重复的部分可以一份交两遍
- 提交的至少是程序源文件，需要说明的部分写在注释里；特别复杂的解释与运行结果的可视化才用 Markdown
- 【方括号包围的作业项目】是【选作】
- 奖励任务在给定期限前提交，可以收获期末额外分数奖励

Q 2 AI 2

奖励任务

- 理解并注释随机数发生器的代码与算法，3 分
  - 只限于 Xorshift, Xoshiro, MT19937_64 三种算法
  - 参考：Numerical Recipes, 3rd Ed.；TAOCP, vol. 2
- kNN 近邻算法，2 分
- Voronoi 剖分算法，1 分
- Dijkstra 最短路径算法，1 分
- 以上算法的并行版本另有加分，小于 4 分
- 花 1 课时为大家讲解 VTune 的用法，5 分
- 花 20 分钟为大家讲解 python profiler，3 分
- 提交期限以课程网站公告为准

AI 3

课程安排

- 理论课：周四，3-5 节，多媒体 322
- 实验课：周四，1-2 节，理 108、110
- 助教：肖竹飞（理 418-2）
- 课程网站：https://git2.taohonker.space
- 作业与考核（两级权重见“考核与评分”一节）
  - 平时成绩 30%：常规作业 60% + **个人 DSA 库 20%** + 甘特图大作业 20%
  - 机试成绩 30%：期中 40% + 期末 60%
  - 期末笔试 40%：闭卷
  - 总评 ≥ 60 分为合格

笔记 1：作业要求

- 每次课上布置，无额外说明时**次周周二晚上**提交，具体时间以课程网站公告为准。
- 作业形式多样，包括编程题、理论题、AI 探究题等。**手写作业需拍照上传**，不要丢弃，期末统一收取。
- 本课程的教学语言是 **C 与 Julia**。题目指定语言的，务必用指定语言完成；题目没有语言要求的，任选 C 或 Julia；使用其它语言导致评分出错的责任自负。
- 编程作业请务必保证代码**可运行**，且输出结果正确，代码风格良好，**有注释**。可使用 Markdown 文件或 Jupyter Notebook 等格式提交展示运行结果，但请**务必包含源代码文件**（可以是 `.c`, `.h`, `.jl`, `.ipynb` 格式）。非必要**请勿附加数据文件与图片**，控制作业体积。
- 作业提交：使用课程 Git 平台（https://git2.taohonker.space）提交。每次作业独立 fork 教师仓库（不改名），本地开发保留完整 Git 记录，导出为独立作业仓库，推送前必须先 pull 同步自己的 fork，再 push。详细规则见课程网站的作业提交说明。提交时有冲突、未按格式提交、未在截止时间前提交均评为 D。未提交不予评分。
- 不要完全依赖 AI 工具，**上机考试时会断网，没有重考机会！**

Q 3 AI 4

考核与评分

理论课（考试课，3 学分）

环节	权重	内容
平时成绩	30%	常规作业 60% + 个人 DSA 库 20% + 甘特图大作业 20%
机试成绩	30%	期中 40% + 期末 60%
期末笔试	40%	闭卷

- **个人 DSA 库**：把课程里讲过的数据结构、基本算法逐个实现、组织成自己的库（接口、测试、复杂度记录都留痕），学期末整体评分。
- 课堂表现是**扣分项、没有基础分**：无故缺课或被公开警告，一次扣 5 分，扣完为止。

详解 1 Q 4 AI 5

实验课（考查课，2 学分）

- 没有环节权重：每个检查点独立按百分制打分，**最终成绩 = 各检查点得分的算术平均**。
- 每个实验交一份报告：目的、过程、结果与分析（复杂度、数据），以及 **AI 使用与验证过程的记录**。分析深度是拿到 90+ 的必要条件。

考试与作业的语言

- 教学语言是 **C 与 Julia**：题目指定语言的按指定，**没有语言要求的任选**。
- 机试禁用直接解题的标准库函数，算法必须手写；只实现题目要求的函数，不需要写 `main`。

斐波那契、特征值与排序

递归算法

斐波那契数列的递归定义

$$F(n) = F(n-1) + F(n-2), \qquad F(1) = 1, \quad F(2) = 1$$

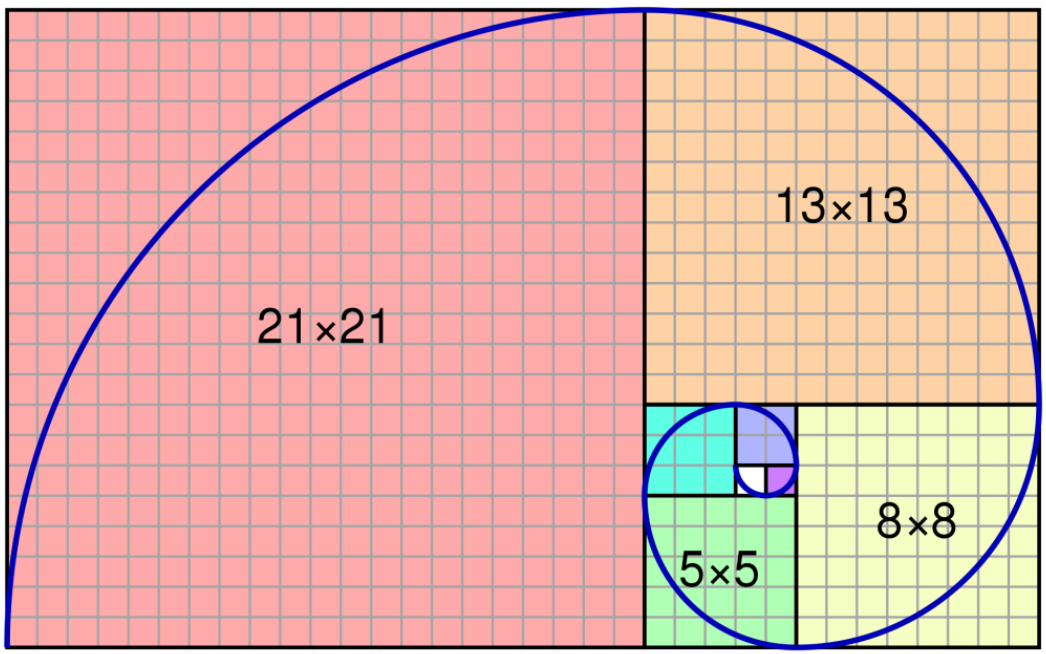


图 1: 斐波那契数列拼成的正方形与黄金螺旋

- 这个递推式是整个学期最简单、也最有用的一个例子：**递归**、**算法复杂性**、**矩阵与特征值**三件事都能在它身上讲完。
- 先用你熟悉的语言把它实现出来，算出 $F(30)$：答案 $F(30) = 832\,040$。

Q 5 AI 6

递归法一：直接照着递推式写

通常用**递归法**计算 $F(n)$ 来衡量一种语言或一种计算芯片的数值性能：代码只有一行，谁来写都一样，因此比出来的就是平台本身。

（本节的计时与画图脚本见 `examples/fib.jl`，四种方法的代码与实测数字都能复现；纳秒级的数字在不同机器与不同次运行之间会有几成浮动。）

```
fib(n) = n ≤ 2 ? 1 : fib(n - 1) + fib(n - 2)

fib(30)      # 832040
@btime fib(30)   # 2.223 ms
```

- 实测 $F(30)$ 要 2.223 ms；换到 $F(45)$ 要 **3.063 s**。
- n 每增加 1，耗时乘约 $\varphi = 1.618$，所以时间是随 n 指数增长的。

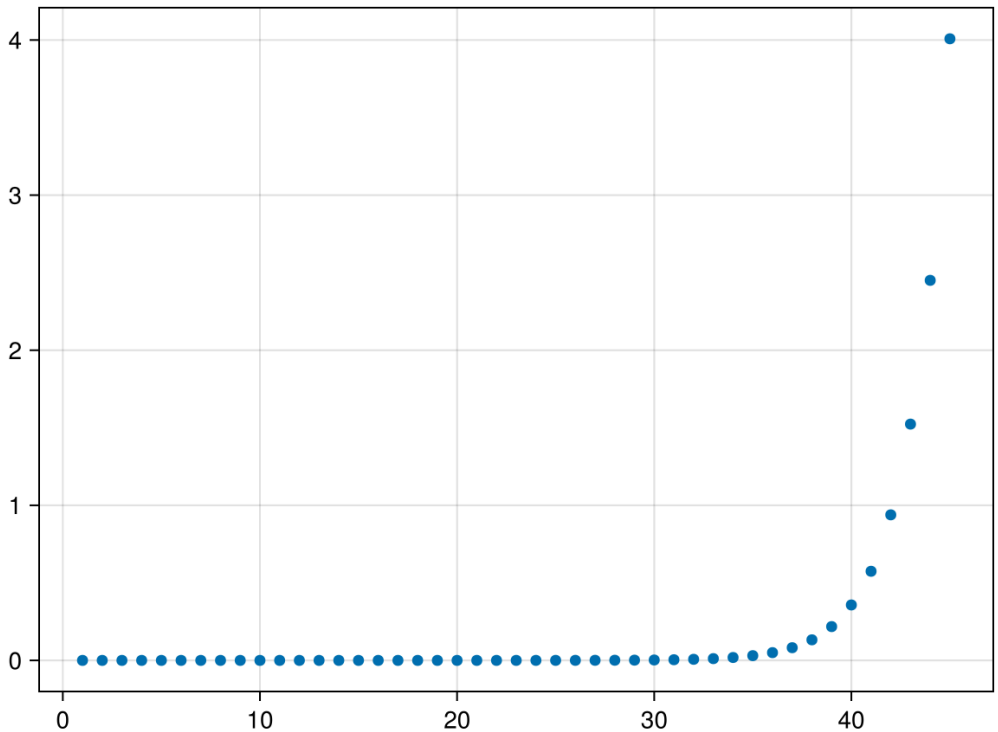


图 2: 递归法一的实测耗时随 n 的变化

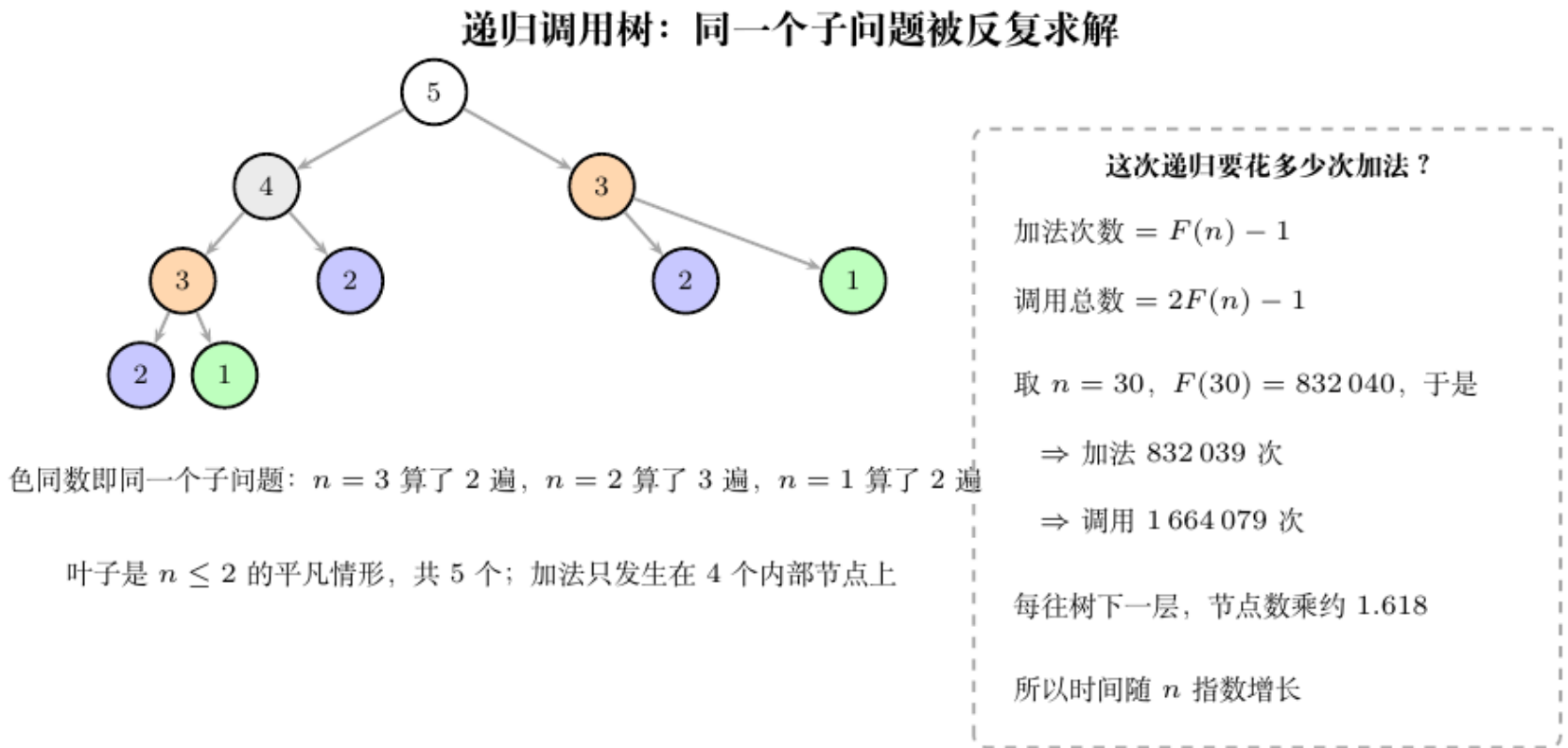


图 3: 斐波那契递归调用的调用树与加法次数

详解 2 Q 6 AI 7

递归法二：递归对

前一种写法的问题在于“一次只要一项”。把两项打包成一个状态：

$$G(n) = (F(n+1), F(n)), \qquad \textbf{于是} \quad F(n) = G(n)_2.$$

（$G(n)_2$ 表示二元组的第 2 个分量，代码里就是 `G(n)[2]`。）这样 $G(n)$ 只依赖 $G(n-1)$，递推关系里不再出现隔项：

$$G(n) = \big(\mathrm{sum}\,(\,G(n-1)\,), G(n-1)_1\big), \qquad G(1) = (1, 1).$$

于是只要算 n 次就能得到 $G(n)$，进而得到 $F(n)$。

```julia
function g(n)
  if n == 1
    return (1, 1)
  end

  tup = g(n - 1)
  return (tup[1] + tup[2], tup[1])
end

fib(45)      # 1134903170, @btime 3.063 s
g(44)        # (1134903170, 701408733)
@btime fib(45)   # 3.063 s
@btime g(44)     # 90.5 ns
```

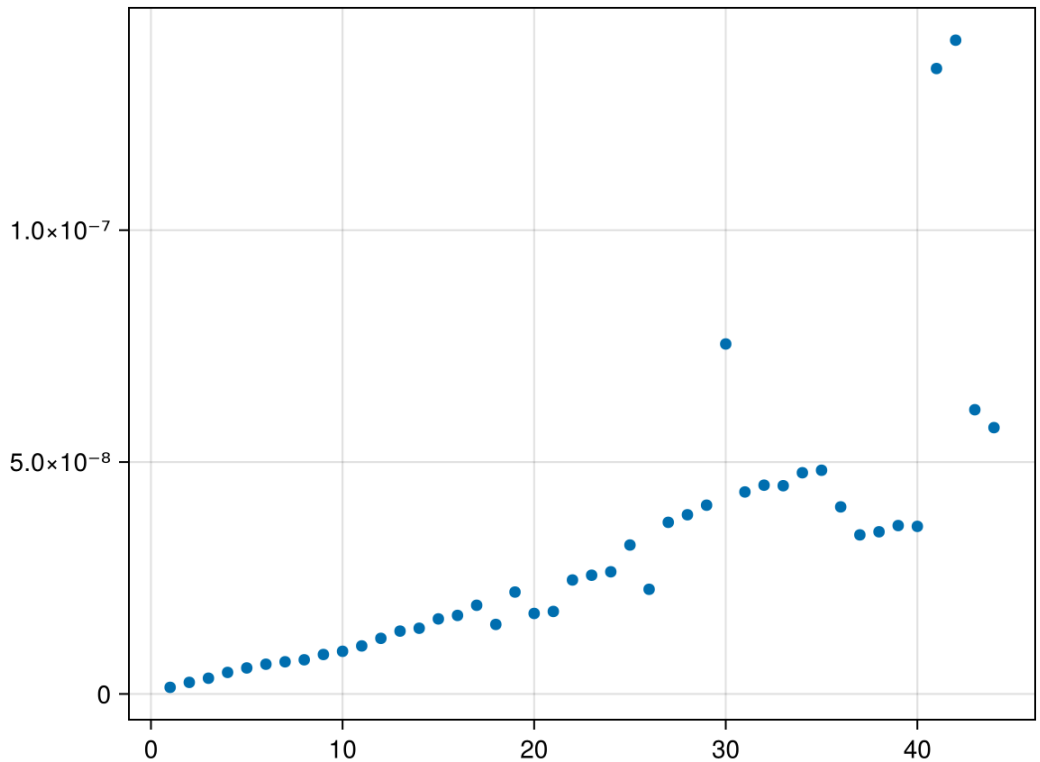


图 4: 递归法二的实测耗时随 n 的变化

- 耗时落在 ns 级别
- 随 n **线性**增长：n 加 1 只多一次函数调用

详解 3 Q 7 AI 8

Fibonacci 数列

数列的封闭式与增长速率

上面两种写法都在做“一步一步算”。那么这个数列本身有没有封闭式，也就是一个直接由 n 算出 $F(n)$ 的公式？有，而且它的两个底数就是黄金比：

$$\varphi = \frac{1+\sqrt{5}}{2} = 1.6180339887\ldots, \qquad \psi = \frac{1-\sqrt{5}}{2} = -0.6180339887\ldots = -\frac{1}{\varphi}$$

$$F(n) = \frac{\varphi^n - \psi^n}{\sqrt{5}}.$$

- 各项之比 $F(n+1)/F(n)$ 随 n 增大趋向 φ，这就是“n 每加 1，工作量乘约 1.618”的来源。
- 因为 $|\psi| < 1$，ψ^n 很快就小到可以忽略：$F(n)$ 就是 $\varphi^n/\sqrt{5}$ 四舍五入到最近的整数。$F(30) = 832\,040$，而 $\varphi^{30}/\sqrt{5} = 832039.9947\ldots$。
- 封闭式把“求第 n 项”变成了“算两个 n 次幂”。这条路要先解决一个问题：φ 与 ψ 是从哪里来的？答案在下一节的**特征值**里。

Q 8 AI 9

精度边界：Double 与 Int64

封闭式里全是浮点运算，这就给它划出了边界。本机实测（julia 1.12，与整数递推逐位对照，取整用 `round`；$G(n)$ 的两个分量分别是 $F(n+1)$ 与 $F(n)$）：

n	$F(n)$ 的偏差	$F(n+1)$ 的偏差
70	0	+1
78	+22	+37
90	+8584	+14523
92	+22595	直接抛 `InexactError`（超出 Int64）

- 偏差每加一层乘约 $\varphi = 1.618$。原因是双精度浮点只有约 16 位有效数字，而封闭式要求把 $\varphi^n/\sqrt{5}$ 四舍五入到整数；$F(n)$ 涨到 10^{15} 以上（$n \geq 70$），这个四舍五入就不再保证正确。
- 换成整数路线就干净得多：递推式、递归对、矩阵幂都用 Int64 整数运算，能逐位精确算到 $F(92) = 7\,540\,113\,804\,746\,346\,429$。
- 所以“用封闭式把算法变成 $O(1)$”这句话要加一个条件：**在双精度还能分辨出整数位之前**。

详解 4 Q 9 AI 10

特征值

矩阵法：把递推写成矩阵幂

把 $G(n) = (F(n+1), F(n))$ 写成矩阵形式，二阶递推就变成了一阶：

$$\begin{pmatrix} F_{n+1} \\ F_n \end{pmatrix} = \begin{pmatrix} 1 & 1 \\ 1 & 0 \end{pmatrix} \begin{pmatrix} F_n \\ F_{n-1} \end{pmatrix}, \qquad G_n = A\,G_{n-1} = A^{n-1} G_1.$$

```julia
using LinearAlgebra
A = [1 1; 1 0]
G1 = [1, 1]
G(n) = A^(n - 1) * G1

G(44)        # 1134903170, 701408733
@btime G(44) # 260.3 ns
```

```julia
using StaticArrays
sA = @SMatrix [1 1; 1 0]
sG1 = @SVector [1, 1]
sG(n) = sA ^ (n - 1) * sG1

sG(44)        # 1134903170, 701408733
@btime sG(44) # 46.3 ns
```

详解 5 详解 6 AI 11

特征值分解

矩阵 $A = \begin{pmatrix} 1 & 1 \\ 1 & 0 \end{pmatrix}$ 的特征值满足 $\lambda^2 - \lambda - 1 = 0$，也就是 $\lambda = \frac{1 \pm \sqrt{5}}{2}$：**它自己的特征值就是黄金比与其共轭**。这解释了封闭式里的 φ 与 ψ 是从哪来的。

两组**特征对**（特征值 + 对应的单位特征向量）实测为：

$$a_1 = -0.6180339887498949 = -\frac{1}{\varphi}, \qquad v_1 = \begin{pmatrix} -0.5257311121191336 \\ 0.8506508083520399 \end{pmatrix},$$

$$a_2 = 1.618033988749895 = \varphi, \qquad v_2 = \begin{pmatrix} 0.85065080835204 \\ 0.5257311121191336 \end{pmatrix}.$$

- 它们满足定义式 $Av_i = a_i v_i$（实测两边之差约 5.6×10^{-17}，即双精度的舍入水平）。
- 两个向量都是单位向量（$|v_i| = 1$）且互相垂直（$v_1 \cdot v_2 \approx 0$）；A 是对称矩阵，不同特征值的特征向量必然正交。
- 特征向量整体乘 -1 仍是特征向量，所以不同软件给出的符号可能不同，这一点在下面的详解里说明。

```
egA = eigen(sA)

v1 = egA.vectors[:, 1]
# -0.5257311121191336, 0.8506508083520399
a1 = egA.values[1]       # -0.6180339887498949 = -1/φ

v2 = egA.vectors[:, 2]
# 0.85065080835204, 0.5257311121191336
```

```
a2 = egA.values[2]       # 1.618033988749895 = φ

l1 = dot(sG1, v1)        # 0.3249196962329063
l2 = dot(sG1, v2)        # 1.3763819204711736
```

详解 7 Q 10 AI 12

特征分析法

有了特征对，矩阵幂就可以拆开：因为 $Av_i = a_i v_i$，所以 $A^{n-1}v_i = a_i^{n-1}v_i$。把 $G_1 = \lambda_1 v_1 + \lambda_2 v_2$ 代入 $G_n = A^{n-1}G_1$：

$$G_n = \lambda_1 a_1^{n-1} v_1 + \lambda_2 a_2^{n-1} v_2.$$

```
1  function GmInt2(n; v1 = v1, v2 = v2, a1 = a1, a2 = a2,
   l1 = l1, l2 = l2)
2    b1 = l1 * a1 ^ (n-1)
3    b2 = l2 * a2 ^ (n-1)
4    fn  = round(Int64, b1 * v1[1] + b2 * v2[1])
5    fn2 = round(Int64, b1 * v1[2] + b2 * v2[2])
6    return fn, fn2
7  end
8
9  GmInt2(44)      # (1134903170, 701408733)
10 @btime GmInt2(44) # 33.3 ns
```

- 33.3 ns，比递归法一的 3.063 s 快约 9×10^7 倍
- 与 n 的关系只剩幂运算内部的对数因子，$n = 1$ 到 $n = 45$ 的实测曲线基本是平的

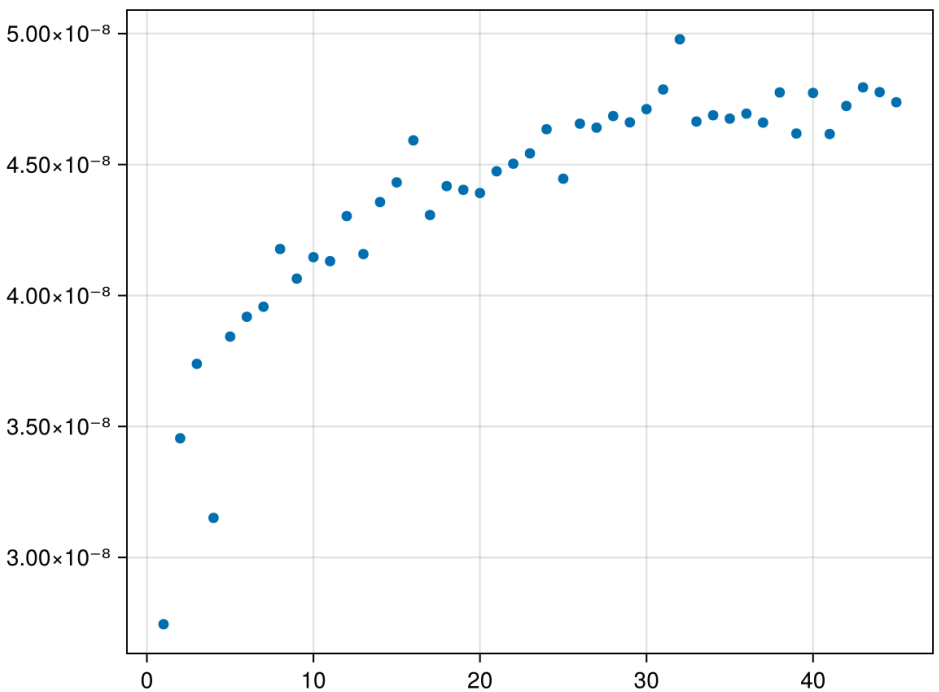


图 5: 特征值法的实测耗时随 n 的变化

- 表面上看问题已经解决了：求第 n 项只要做两次幂运算，算法的时间复杂性被搬进了**数值计算**里。

- 取整必须用 `round`，不能用 `floor`：重建值带约 10^{-16} 的相对误差，n 小的时候它会落在整数下方（实测 $n = 1$ 时重建值是 0.9999999999999999，`floor` 给出 0，`round` 给出 1）。
- 代价是上一节量出来的精度边界（n 到 70 之后结果不再逐位正确），以及“常数要在 n 确定前先算好”这一步。

特征分析法

笔记 2：扩展学习

1. 学习 IEEE-754 标准关于数值算法的规定
2. Float64 的精度上限比 Int64 低
3. 学习 / 复习如何画图
    i. Julia: Makie.jl
    ii. Python: Matplotlib
    iii. MatLab
    iv. GNUPlot

重要 1：程序慢的可能来源

- 算法不行！
- 数据结构不行！
- 语言不行！
- 机器不行！
- 客户不行！
- 程序员不行！
- AI 不行！

AI 13

排序算法

排序：一条新闻

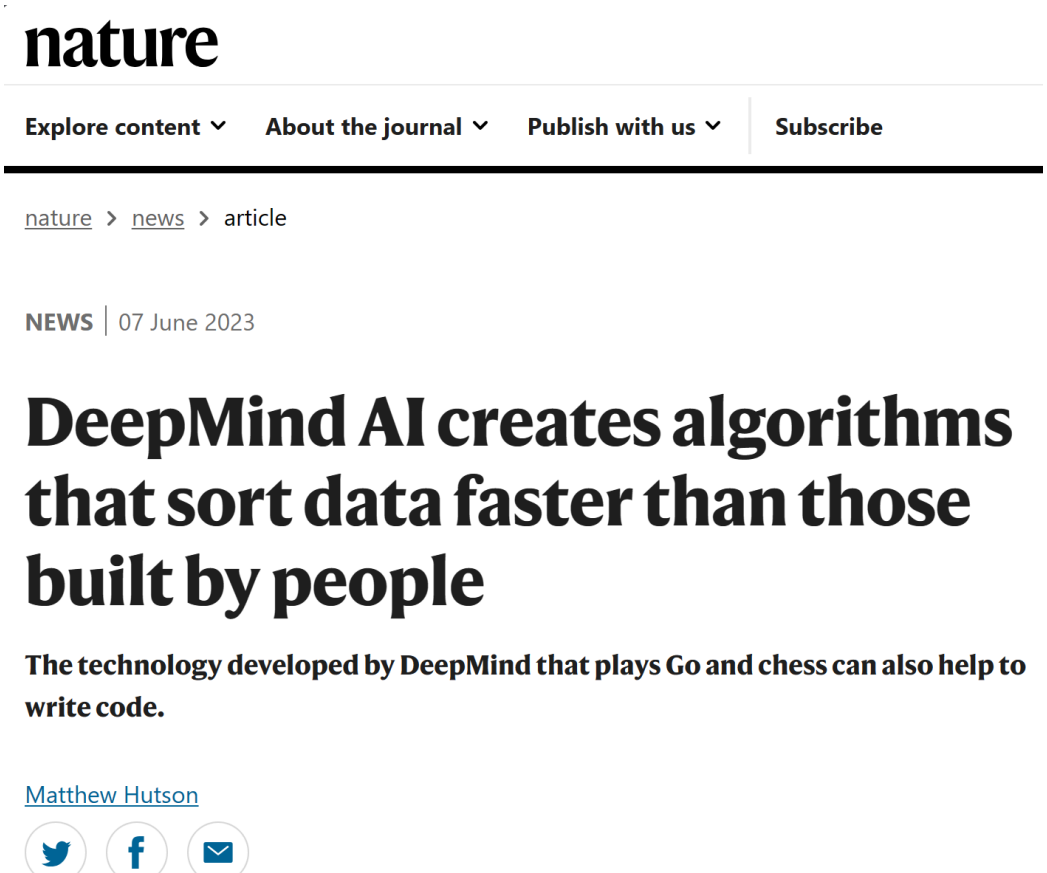
一条新闻

- AlphaDev（2023，DeepMind）：让 AI 从零合成排序代码，成果进了 LLVM 的标准库
  - 对短序列（3 到 5 个元素）排序，在汇编层面发现新的优化方式
  - 某些步骤不再用 `MAX(A,B,C)`，而是拆成 `MAX(A,B)` 的组合
  - 论文报告：短序列快约 70%，长序列（数十万级）约 1.7%
- 这条新闻说明了排序的两个侧面：**要理解算法本身**，也要理解它在具体机器上的常数。

参考

- TAOCP, vol. 3
- Algorithms, 4th Ed., Ch2
- Sorting Algorithms Animations

Q 11 AI 14



nature

Explore content ⌄ About the journal ⌄ Publish with us ⌄ Subscribe

nature › news › article

NEWS | 07 June 2023

DeepMind AI creates algorithms that sort data faster than those built by people

The technology developed by DeepMind that plays Go and chess can also help to write code.

Matthew Hutson

图 6: 《自然》对 AlphaDev 排序算法的报道截图

冒泡排序



图 7: 冒泡排序各轮扫描的过程示意

（完整脚本：`examples/bubble_sort.jl`。）

```julia
function bubble_sort!(v::AbstractVector)
  tmp = zero(eltype(v))
  len = length(v)
  for i in 1:len
    for j in 1:len-i
      if v[j] > v[j+1]
        tmp = v[j]
        v[j] = v[j+1]
        v[j+1] = tmp
      end
    end
  end
  return nothing
end
```

```julia
using BenchmarkTools

v = rand(10000)
bubble_sort!(v)
issorted(v)   # true

@btime bubble_sort!(rand(10000)) # 44.7 ms
@btime bubble_sort!(rand(100))   # 9.06 μs

@btime sort!(rand(10000)) # 128.5 μs
@btime sort!(rand(100))   # 1.61 μs
```

详解 8 Q 12 AI 15

插入排序

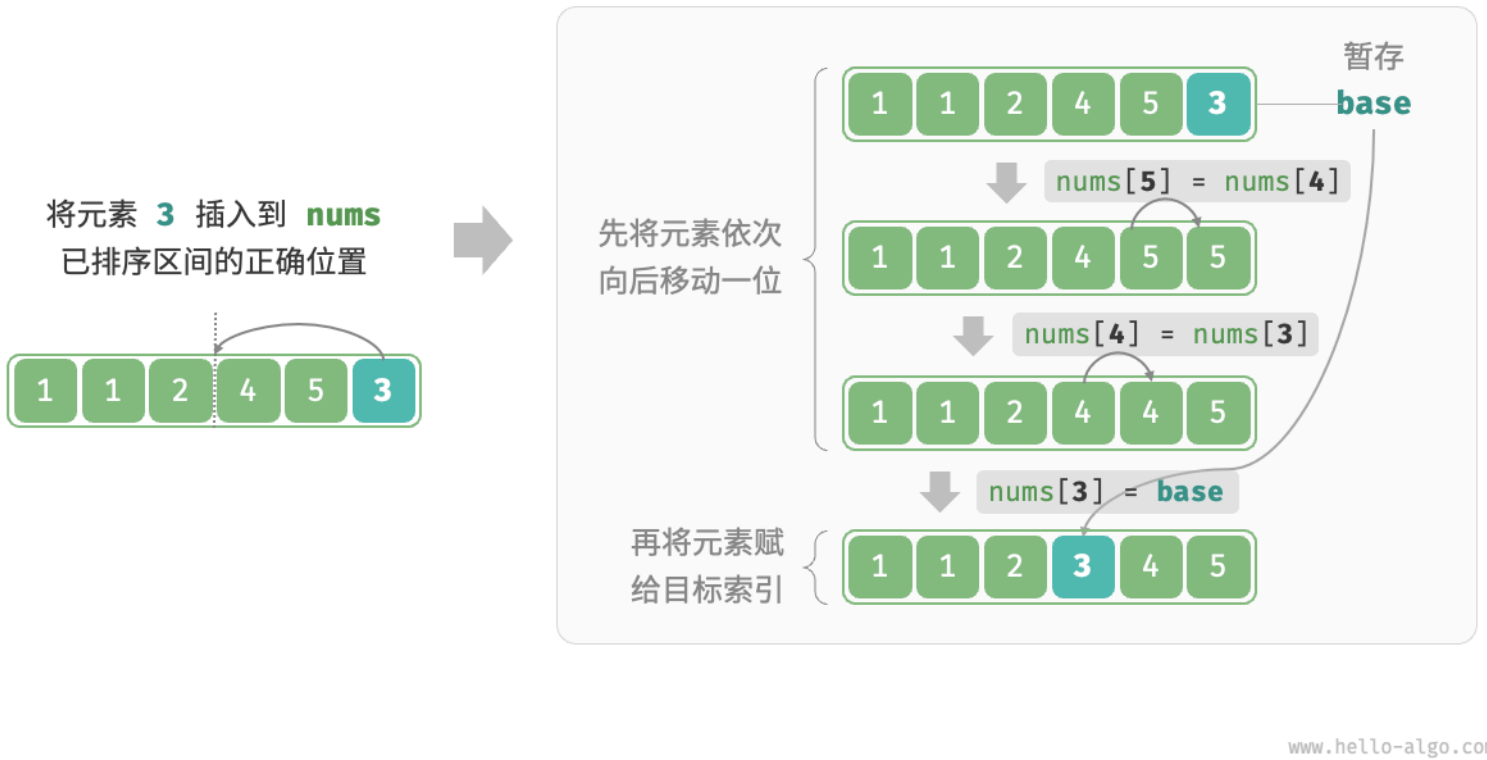


图 8: 插入排序一次插入的三步操作

（完整脚本：`examples/insertion_sort.jl`。）

```julia
function insertion_sort!(v::AbstractVector)
  len = length(v)
  for i in 2:len
    j = i
    x = v[i]
    while j > 1
      y = v[j-1]
      if x > y
        break
      end
      v[j] = y
      j -= 1
    end
    v[j] = x
  end
  return v
end
```

```julia
using BenchmarkTools

v = rand(1:100, 10000)
insertion_sort!(v)
issorted(v)   # true

@btime insertion_sort!(rand(10000)) # 9.5 ms
@btime insertion_sort!(rand(1000))  # 98.6 μs
@btime insertion_sort!(rand(100))   # 2.09 μs

@btime sort!(rand(10000)) # 128.4 μs
@btime sort!(rand(1000))  # 8.64 μs
@btime sort!(rand(100))   # 1.58 μs
```

详解 9 Q 13 AI 16

快速排序

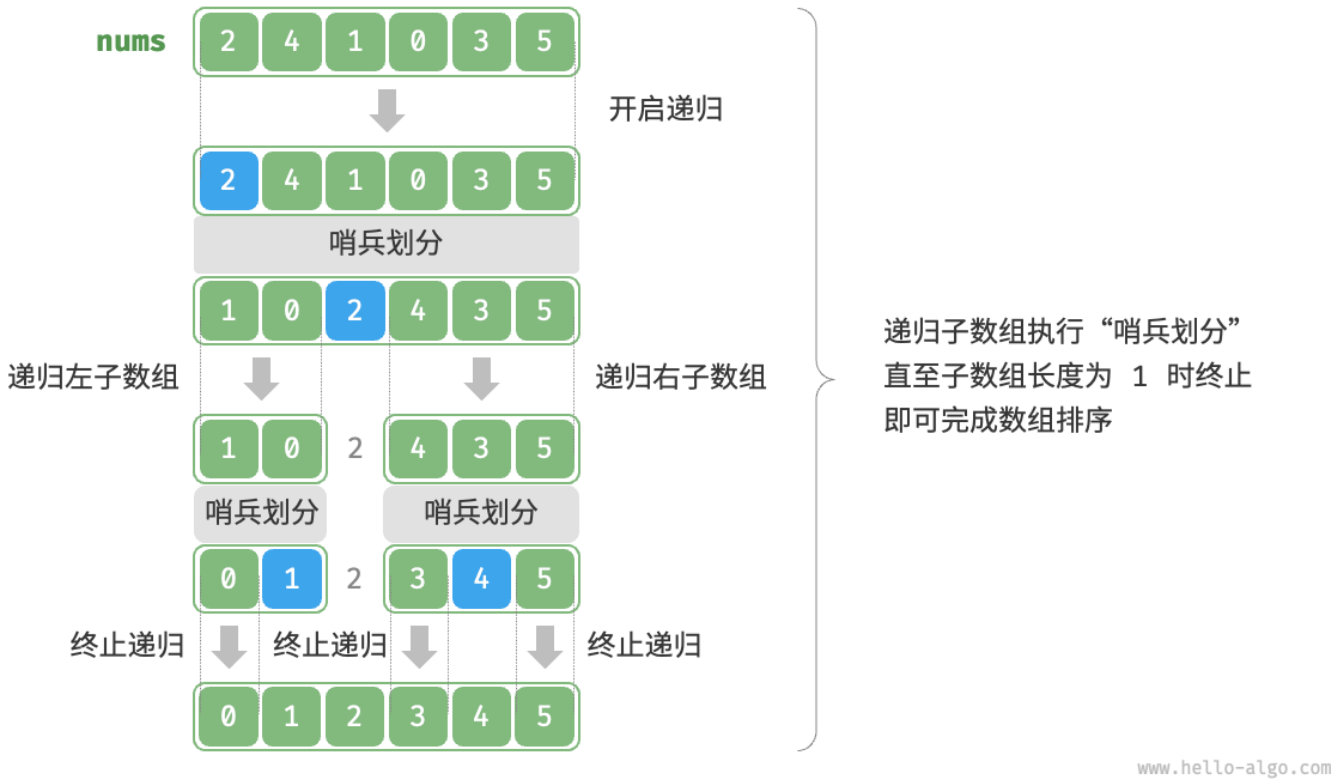


图 9: 快速排序以哨兵划分的分治流程

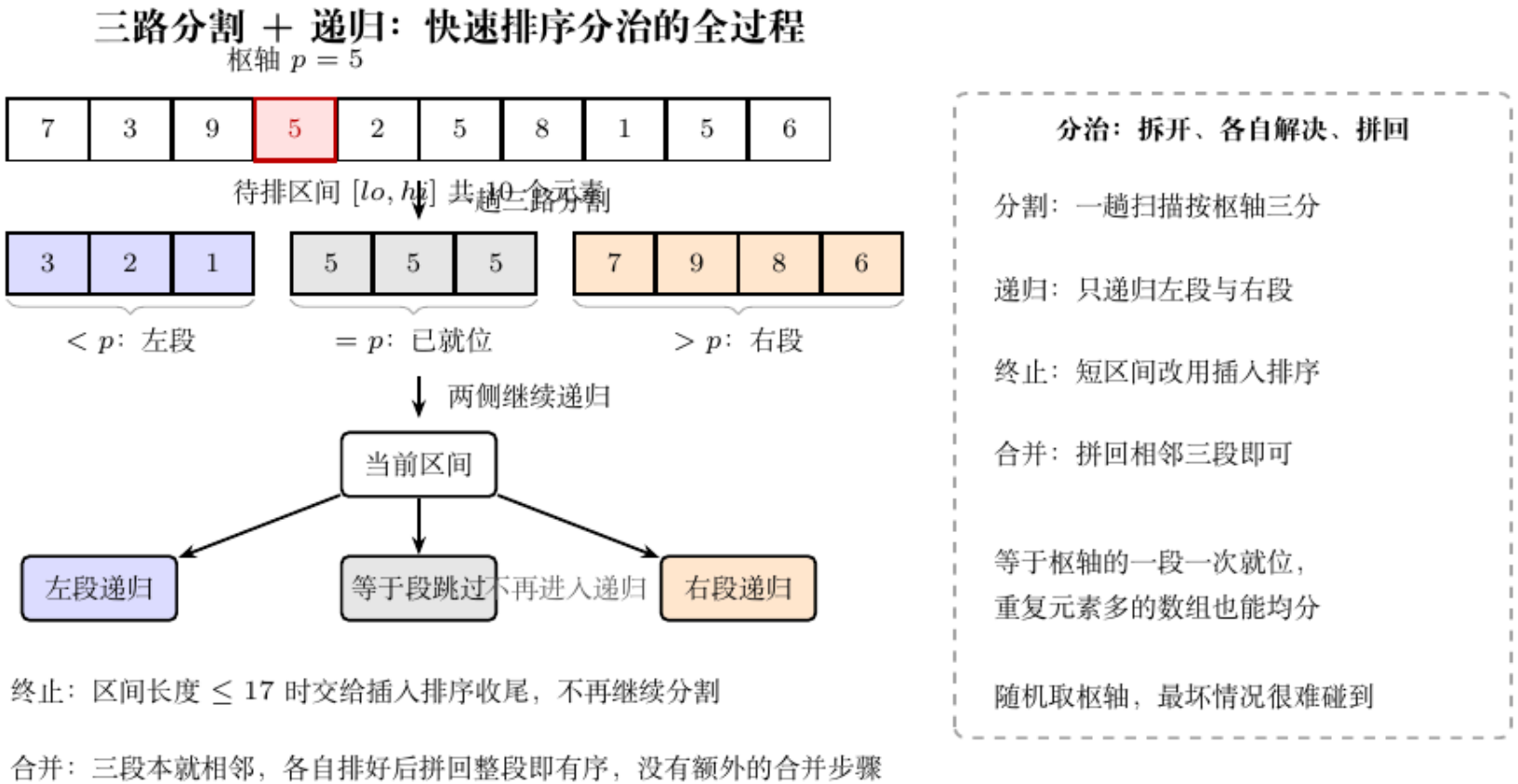


图 10: 快速排序三路分割与递归的分治过程

接口：对外只留一个函数，内部另分配一块同样大的辅助数组 t。

```
function quicksort!(v::AbstractVector)
  lo = 1
  hi = length(v)
```

```
  t = copy(v)
  quicksort!(v, lo, hi, t)
  return v
end
```

快速排序

分治：区间还长的时候分割成两半递归，短到一定程度就交给插入排序收尾。

```
function quicksort!(v::AbstractVector, lo, hi, t::AbstractVector)
  if hi - lo ≤ 16
    vv = @view v[lo:hi]
    insertion_sort!(vv)
    return nothing
  end

  lo1, hi1, lo2, hi2 = partition!(v, lo, hi, t)
  if lo1 < hi1
    quicksort!(v, lo1, hi1, t)
  end
  if lo2 < hi2
    quicksort!(v, lo2, hi2, t)
  end
  return nothing
end
```

快速排序

3-路分割：一趟扫描把区间按枢轴分成三段，等于枢轴的一段留在中间。分段时只用一块与区间等长的辅助数组 t：小于枢轴的从头部往后放，大于枢轴的从尾部往前放，等于枢轴的先不落位、最后整段填回去。

```
function partition!(v::AbstractVector, lo, hi, t::AbstractVector)
  ngreater = zero(lo)
  nequal = zero(lo)
  myone = one(lo)
  pivot_index = rand(lo:hi)   # 随机取枢轴，避免在已排序输入上退化
  pivot = v[pivot_index]

  @inbounds begin
    for i in lo:hi
      x = v[i]
      if x < pivot
        t[i-ngreater-nequal] = x
      elseif x > pivot
        t[hi-ngreater] = x
        ngreater += myone
      else
        nequal += myone
      end
    end
    t[hi-ngreater-nequal+myone:hi-ngreater] .= pivot
```

```
    v[lo:hi] .= t[lo:hi]
  end

  return lo, hi - ngreater - nequal, hi - ngreater + myone, hi
end
```

快速排序

```
using BenchmarkTools
# Float64
@btime quicksort!(rand(10000)) # 0.553 ms
@btime quicksort!(rand(1000))  # 42.4 μs
@btime quicksort!(rand(100))   # 2.77 μs
@btime sort!(rand(10000)) # 128.5 μs
@btime sort!(rand(1000))  # 8.55 μs
@btime sort!(rand(100))   # 1.57 μs
# Int64（取值范围只有 1:100，重复元素很多）
@btime quicksort!(rand(1:100, 10000)) # 251.3 μs
@btime quicksort!(rand(1:100, 1000))  # 30.3 μs
@btime quicksort!(rand(1:100, 100))   # 2.63 μs
@btime sort!(rand(1:100, 10000)) # 26.8 μs
@btime sort!(rand(1:100, 1000))  # 3.65 μs
@btime sort!(rand(1:100, 100))   # 596 ns
```

详解 10 Q 14 AI 17

三种排序的性能对比

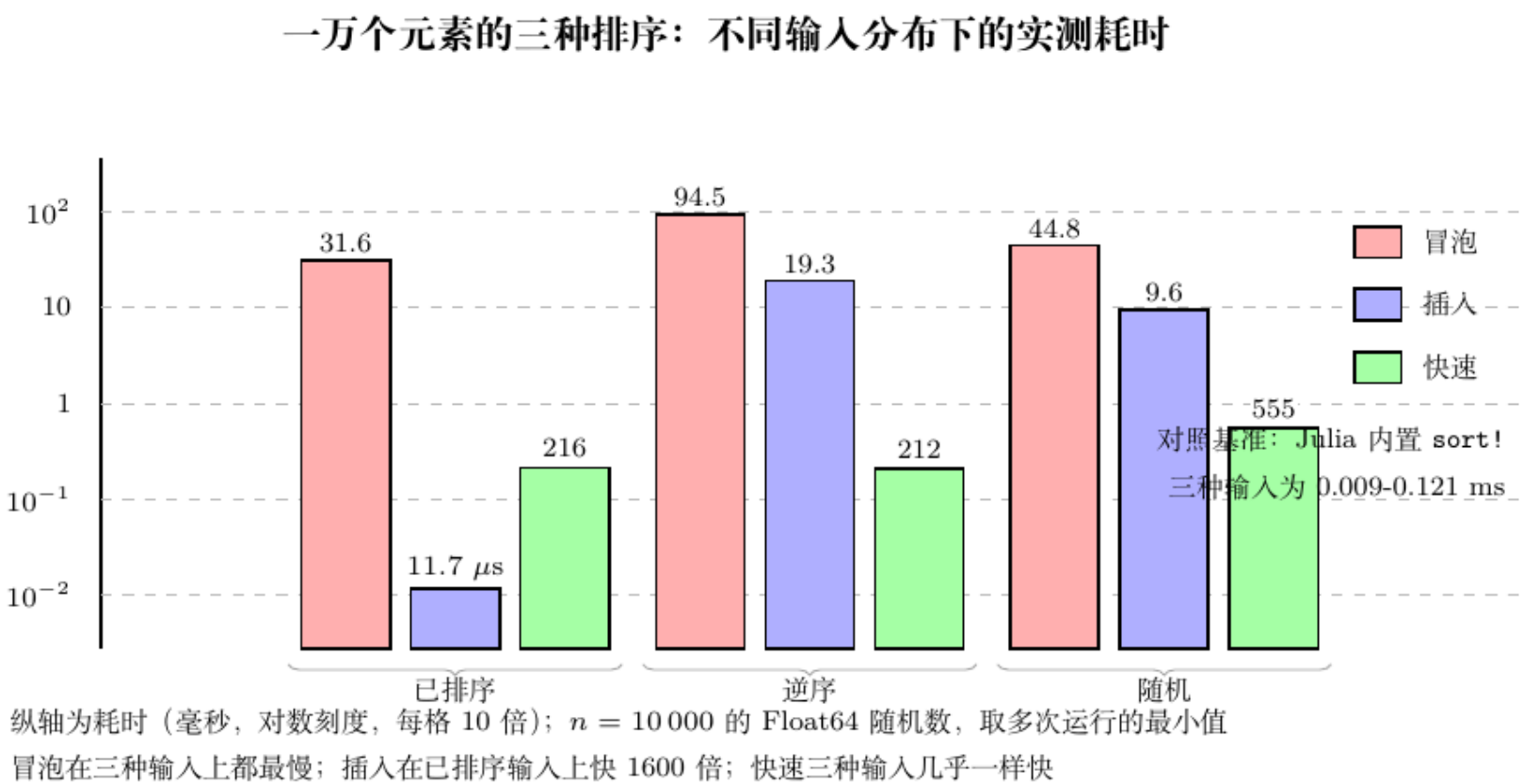


图 11: 三种排序在三种输入分布下的实测耗时

同样的 $n = 10000$、同样的机器，只换输入分布（单位：ms；每个数字取多次运行的最小值，同一台机器上重复测量的抖动在几个百分点以内）：

算法	已排序	逆序	随机
冒泡排序	31.6	94.5	44.8
插入排序	0.0117	19.25	9.64
快速排序（3-路）	0.216	0.212	0.555
Julia 内置 `sort!`	0.00888	0.01163	0.121

三句话总结这张表：

- **冒泡排序**在三种输入上都最慢。它的比较次数固定，且每次只交换相邻的两个元素，数据移动多。
- **插入排序**的结果完全看输入：已排序输入上每个元素只比较一次，比 `sort!` 只差一点；逆序输入上退化成平方级，比已排序输入慢 1600 倍。
- **快速排序**在三种输入上都很快，随机输入比已排序输入慢约 2.6 倍；两种输入的分割次数相近，差别在访存与分支预测上（已排序的数据是顺序访问，随机数据不是）。

详解 11 详解 12 Q 15 AI 18

作业

作业要求

- 本次作业使用 C 语言或 Python + C 的混合方式完成，也可以使用 Julia 语言完成
- 截止时间：无额外说明时为**次周周二晚上**，具体以课程网站公告为准
- 提交方式、命名与格式要求见“课前”一节的作业要求；**必须包含可运行的源代码文件**

Q 16 AI 19

作业一：四种斐波那契算法

使用 C 与 Python 分别实现 Fibonacci 序列的 4 种计算方法：

1. 递归法一（直接照着递推式写）
2. 递归法二（递归对）
3. 矩阵法（$G_n = A^{n-1}G_1$）
4. 特征值法（封闭式）

并作图：

- 比较两种语言中递归法一的速度
- 比较 Python 中递归法一与特征值法的速度

Q 17 AI 20

作业二：C 函数库与 Python 调用

使用 C 语言实现下列算法，制作**函数库**：

1. `issorted()` 函数检查有序性
2. 冒泡法
3. 插入排序
4. 3-路快速排序

使用 Python 从 C 编译的函数库中**调用** 4 种算法，测试它们对大（$n = 10^6$）、小（$n = 20$）随机数组的性能差异。

【还可以比较用 Python 实现的排序算法的速度，以及 Python 标准库的排序算法的速度。】

Q 18 AI 21

作业三：用指针数组搭一个上三角矩阵

这一次专门练 **C 的指针与动态内存**。要把一个上三角矩阵存成“每行一段连续数组、行与行用指针数组串起来”的结构。

要求

1. 只存非零部分：第 i 行存第 i 到第 n 列的元素，行长度随行号递减（第 1 行 n 个、第 2 行 $n-1$ 个……最后一行 1 个），总共 $n(n+1)/2$ 个元素。
2. 行与行之间用**指针数组**组织，例如

```c
typedef struct {
  int n;          /* 阶数 */
  double **rows;  /* rows[i] 指向第 i 行的数组，长度 n-i */
} UpperTri;
```

   不允许用二维数组 `double a[n][n]`，也不允许用变长数组（VLA）。
3. 实现 `create(n)`、`destroy(a)`、`get(a,i,j)`、`set(a,i,j,v)`，下标从 0 开始，$i < j$ 的位置（下三角）一律视为 0、不占存储。
4. 实现三个乘法函数，并说明结果是什么结构：
   - 上三角 × **上三角**：结果仍是上三角，请只算需要的部分；
   - 上三角 × **下三角**：结果一般是普通方阵，请完整分配存储；
   - 上三角 × **普通矩阵**：结果一般是普通矩阵。
5. 做**复杂度对比**：把“利用三角结构只算需要的部分”与“先补成普通矩阵再做 $O(n^3)$ 相乘”两种做法，分别统计乘加次数，并实测 $n = 100, 500, 1000$ 的耗时，给出结论。

6. 做**正确性验证**：把你的三种乘法结果与普通矩阵乘法的结果逐元素对照（误差 $< 10^{-12}$），并验证 $(AB)^T = B^T A^T$ 这类恒等式。
7. **内存自证**：`destroy` 之后不能有泄漏。用 `valgrind --leak-check=full` 跑一遍，或者自己在 `malloc/free` 处计数并打印配平结果。

【选作】把同一个结构再用双向链表实现一遍（每行一个结点，结点里存行数组指针），比较两种实现在随机访问与遍历上的差别。

Q 19 AI 22

参考资料

参考资料

课程讲义尽可能做到自足，有问题请及时提问、反馈，找到错误会有奖励。

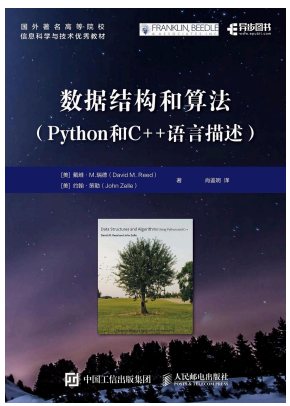
参考资料：

- 互动精致开源教材：靳宇栋，Hello, 算法！
- 国内新近高质量教材：俞勇 等，《数据结构》（101 计划）
- 国内优秀数据结构教材：邓俊辉，《数据结构（C++语言版・第 3 版）》[1]
- 国外优秀算法入门：A. Levitin, 《算法设计与分析基础》
- 国外优秀数据结构与算法：D. Reed, J. Zelle, 《数据结构和算法（Python 和 C++ 语言描述）》
- 经典算法教材：T. Cormen 等, 《算法导论（原书第 3 版）》[2]









Q 20 AI 23

[1]清华大学，邓老师的书与授课材料包括视频均由作者本人在其主页免费提供，强烈推荐！

[2]算法圣经，内容全面，适合有一定基础的同学深入学习。非常难！英文版已出版 4th edition, 中文版译自 3rd edition.